

Reinforcement Learning

Class 3

Planning – Dynamic Programming for known MDP

Mostly based on D. Silver (UCL) lecture 3 from 2015
and chapter 4 from Sutton & Barto's book

Context

- Last lecture – we presented MDPs as the mathematical framework underlying RL problems (but we did not solve them)
- This lecture – we will study how to solve a **known MDP** using Dynamic Programming
 - Policy evaluation (known policy, predict value function)
 - Policy iteration (improving policies)
 - Value iteration (alternative way, iterating value functions)
- Next lectures - we will study methods to handle unknown MDPs, by estimating and optimizing value functions

Dynamic programming: what is this ?

- **Dynamic** – implies a sequential/ temporal component to the problem
- **Programming** – Mathematical optimization
- DP is a class of methods to solve different complex problems, when it is possible to **break them into sub-problems** and to **combine** their solutions to obtain the solution of the original problem
- It is a deterministic method that allows reaching **optimal solutions** (expressed recursively in terms of optimal subsolutions)

Requirements for Dynamic Programming

- The **Principle of Optimality** applies - **optimal solution** can be **decomposed** into subproblems
- Overlapping subproblems that **recur many times** and whose solutions can be saved and **reused**

MDPs satisfy both these properties:

- **Bellman equation** gives **recursive** decomposition
- **Value function** stores and reuses solutions

Applications of dynamic programming

DP is used to solve problems in diverse areas:

- Scheduling/ production planning, inventory (e.g. in industry)
- String algorithms (e.g. finding similarity in texts, string distances, longest common etc)
- Bioinformatics (sequence alignment, etc)
- Graph algorithms (e.g. shortest path algorithms such as Bellman – Ford, Floyd-Warshall)
- Hidden Markov models and other graphical models (e.g. Viterbi algorithm)
- ...

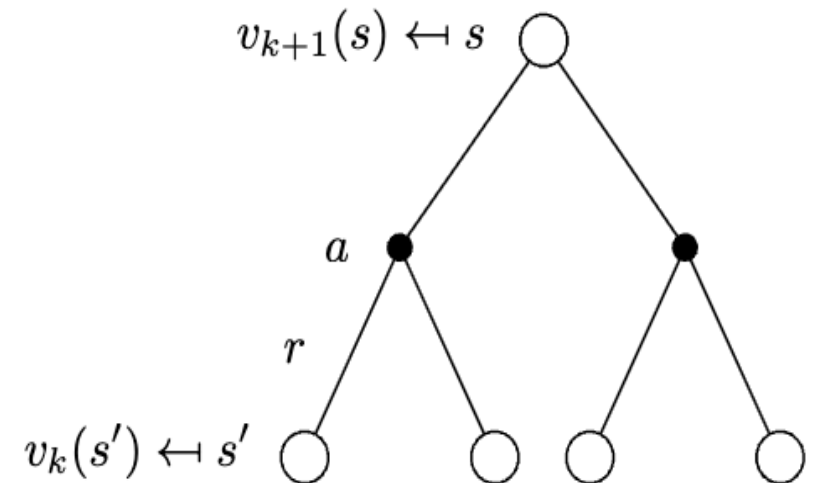
Bellman is a recurring name in these classes – it refers to Richard Bellman, the founding father of DP

Planning by Dynamic Programming

- Dynamic programming assumes **full knowledge of the MDP**, thus it is used for **planning**
- For **prediction**:
 - Inputs: the **MDP** $\langle S, A, R, P, \gamma \rangle$ and a **policy** π
 - Output: the **value function** for that policy v_π
- For **control** (to **solve** the MDP):
 - Input: the **MDP** $\langle S, A, R, P, \gamma \rangle$
 - Output: **optimal value function** v_* and **optimal policy** π_*

Iterative policy evaluation - prediction

- Problem: evaluate a given policy π
- Solution: iterative application of Bellman **expectation** backup
- To generate a sequence of estimates of value functions $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow \mathbf{v}_\pi$
- At each iteration $k+1$ and for all states s in S (synchronous updates): we will update $v_{k+1}(s)$ from $v_k(s')$ where s' is the successor state of s
- Convergence to v_π can be proven if $\gamma < 1$ or termination is guaranteed from all states (it is a fixed point)



$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

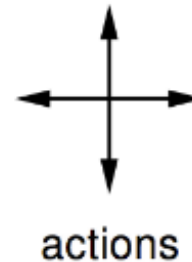
Vector form: $\mathbf{v}^{k+1} = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}^k$

Iterative policy evaluation

- Applies the same operation to each state s : replaces the old value of s with a new value obtained from the old values of the successor states of s , and the expected immediate rewards, along all the one-step transitions possible **under the policy being evaluated**.
- We call this an ***expected update*** - based on an expectation over all possible next states rather than on a sample next state
- We keep two arrays for old and new values for v_k , the new values can be computed one by one without the old values being changed (**synchronous**); **in place** versions can be used to replace the values immediately as they are available
- **Termination** is, in practice, achieved once the changes are very small (we are near the fixed point); the maximum update for all the states is calculated in each iteration and the process starts when this is small than a given threshold (e.g. 10^{-8})

Example - gridworld

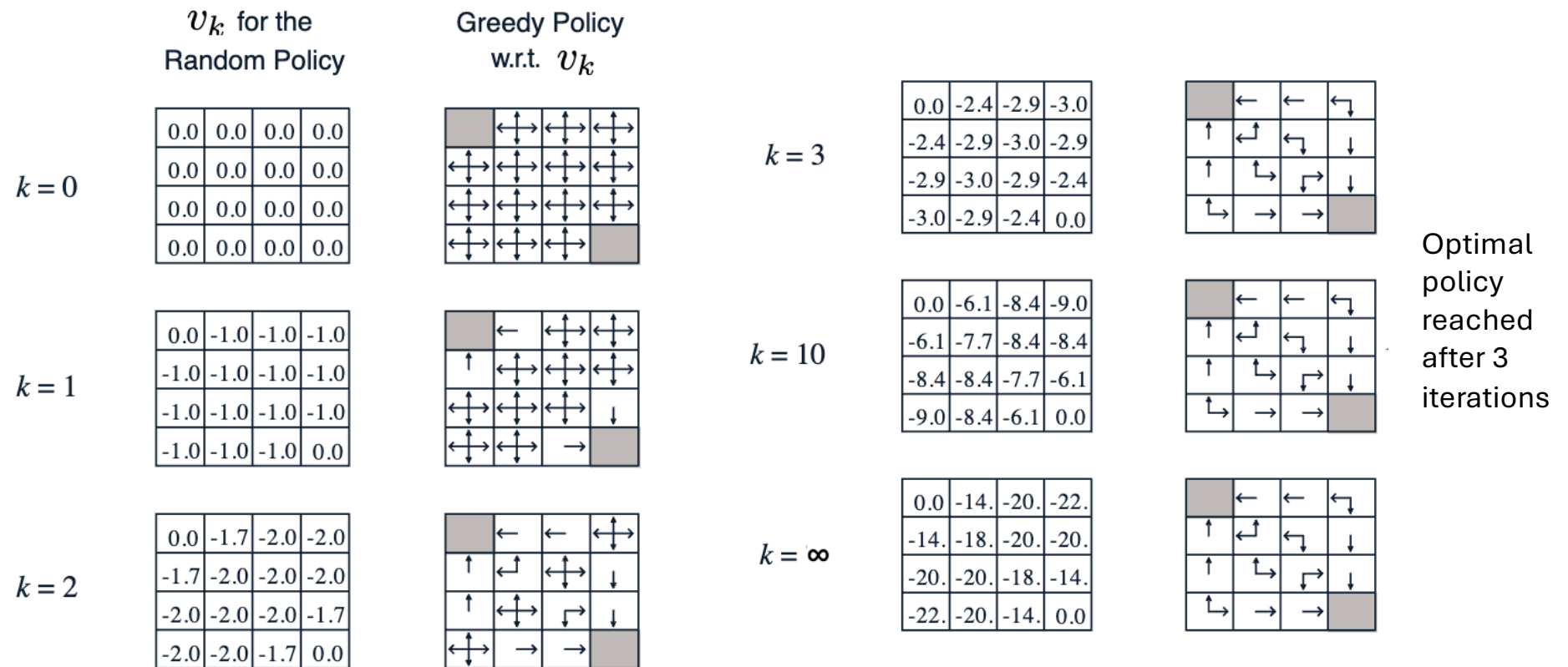
- Terminal state - in the corners (0,0), (3,3) - shaded
- Non-terminal states (1, ..., 14)
- Rewards equal to -1 on all transitions, except from terminal states to themselves
- Actions: up, down, left, right
- Actions leading out of the grid leave state unchanged (but still get -1 reward)
- Policy – uniform random



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$$\pi(n|\cdot) = \pi(e|\cdot) = \pi(s|\cdot) = \pi(w|\cdot) = 0.25$$

Example – iterative policy evaluation



Simulation with undiscounted scenario

Improving policies – policy iteration

- Our reason for computing the value function for a policy is to help find better policies.

- Given a policy π

- **Evaluate** the policy π
(liked shown before)

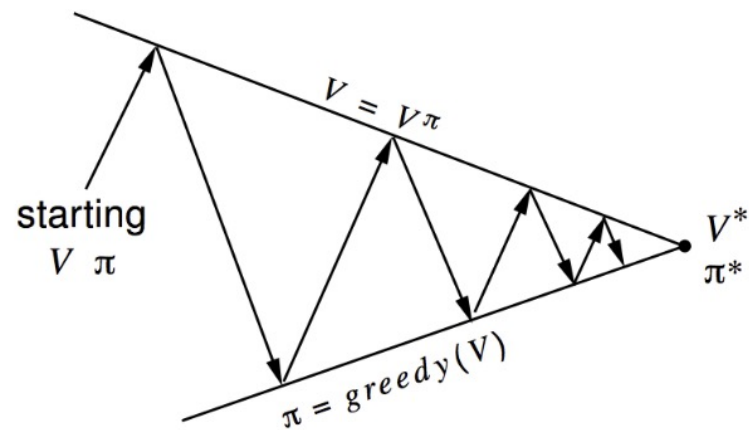
$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi' = \text{greedy}(v_{\pi})$$

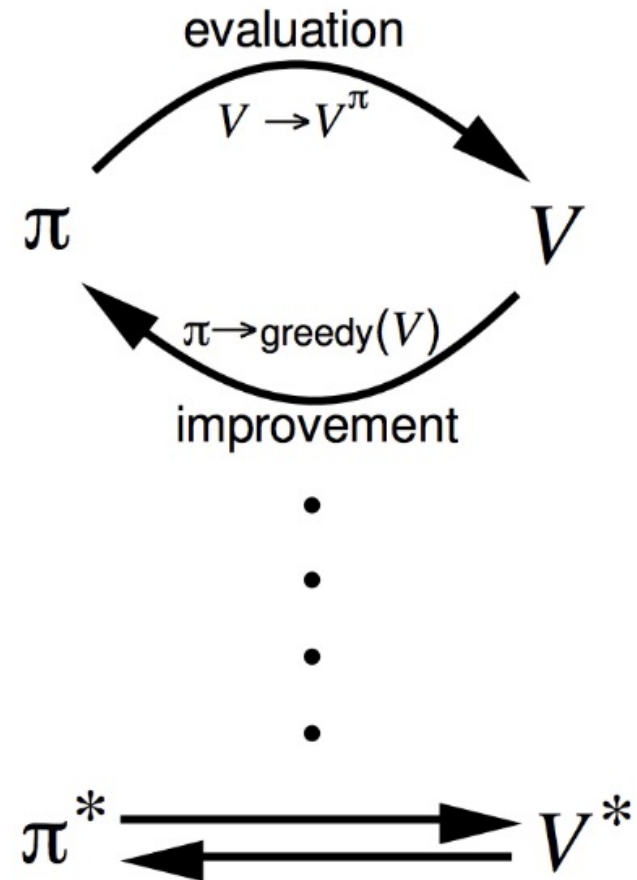
- In the Small Gridworld, the improved policy was optimal $\pi' = \pi_*$
- In general, need more iterations of improvement / evaluation
- But this process of **policy iteration** always converges to optimal π_*

Policy iteration



Policy **evaluation** – estimate v_π –
iterative policy evaluation

Policy **improvement** – generate $\pi' \geq \pi$ –
greedy policy improvement



Policy improvement

- Consider a deterministic policy $\pi(s)$
- We can improve the policy by acting greedily $\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} q_{\pi}(s, a)$
- This improves the value from any state s over one step

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- It therefore improves the value function, $v_{\pi'}(s) \geq v_{\pi}(s)$

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s] = v_{\pi'}(s) \end{aligned}$$

Policy improvement

- If improvements stop

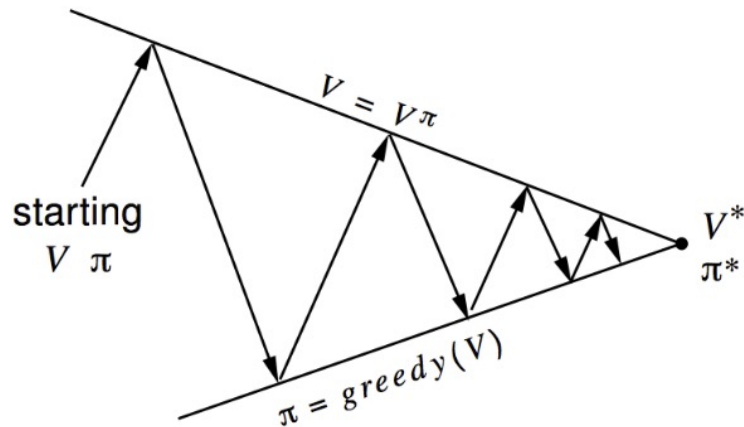
$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- Then the Bellman optimality equation has been satisfied

$$v_{\pi}(s) = \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

- Therefore $v_{\pi}(s) = v_{*}(s)$ for all $s \in S$, and so **π is an optimal policy**
- This is valid for deterministic policies, but these results extend to stochastic policies

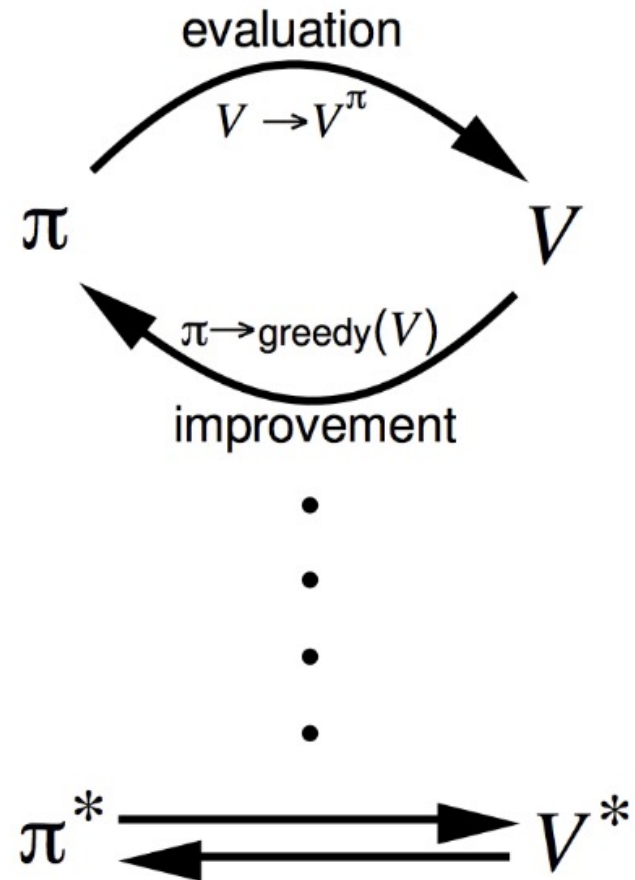
Generalized Policy iteration



$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

E- Policy **evaluation** – estimate v_{π} –
any policy evaluation algorithm

I - Policy **improvement** – generate $\pi' \geq \pi$ -
any policy improvement algorithm



Policy iteration – full algorithm

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow true$

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow false$

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Pseudo-code from
Sutton & Barto

Note that the transition
function notation is slightly
distinct

Principle of optimality

Any optimal policy can be subdivided into two components:

- An optimal first action A_*
- Followed by an optimal policy from successor state s'

Theorem – *Principle of Optimality*

A policy $\pi(a|s)$ achieves the optimal value from state s , $v_\pi(s) = v_(s)$, if and only if*

- *For any state s' reachable from s
 π achieves the optimal value from state s' , $v_\pi(s') = v_*(s')$*

Modified policy iteration

- A problem with the previous solution is that each iteration involves policy evaluation, itself an iterative process
- Does policy evaluation need to converge to v_π ?
- Or should we introduce a stopping condition ?
- Or simply stop after k iterations of iterative policy evaluation?
- For example, in the small gridworld, $k = 3$ was sufficient to achieve optimal policy
- Why not update policy every iteration? i.e. stop after $k = 1$
 - This is equivalent to value iteration (next)

Deterministic value iteration

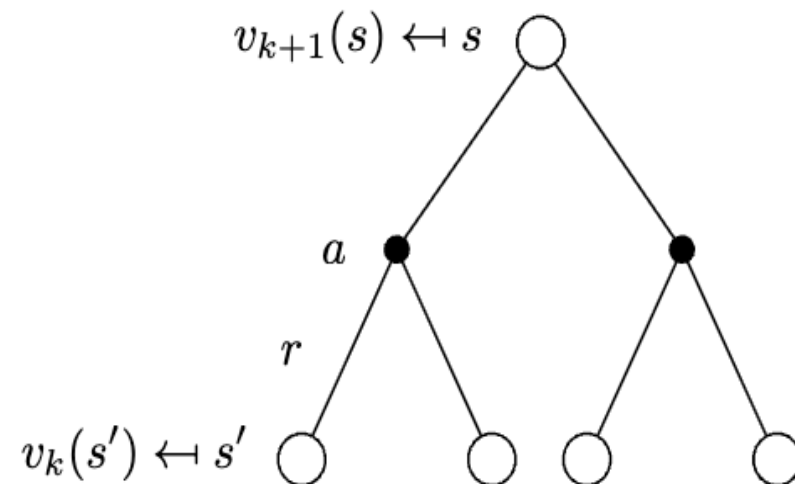
- If we know the solution to subproblems $v_*(s')$, then solution $v_*(s)$ can be found by one-step lookahead (**Bellman optimality equation**)

$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

- The idea of **value iteration** is to apply these **updates** iteratively (turns Bellman equation into update rule)
- Can be thought as a simple update that combines the policy improvement and truncated policy evaluation (one step)
- Intuition: start with final rewards and work **backwards**
- Works with MDPs with loops and stochastic MDPs

Value iteration

- Problem: find **optimal policy** π
- Solution: iterative application of Bellman **optimality** backup
- Generating a sequence of value functions $v_1 - v_2 - \dots - \mathbf{v}_*$
- At each iteration $k+1$ and for all states s in S : we will update $v_{k+1}(s)$ from $v_k(s')$
- Unlike policy iteration, there is **no explicit policy**
- Intermediate value functions may not correspond to any policy, but it converges the optimal policy



$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$
$$\mathbf{v}_{k+1} = \max_{a \in \mathcal{A}} \mathcal{R}^a + \gamma \mathcal{P}^a \mathbf{v}_k$$

Value iteration - algorithm

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Pseudo-code from
Sutton & Barto

Note again that the
transition function notation
is slightly distinct

Example: shortest path

Rewards the same as grid world
Just one terminal state
Now showing **value iteration**

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Summary of DP algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- Algorithms are based on state-value function $v_{\pi}(s)$ or $v_{*}(s)$
- Complexity $O(mn^2)$ per iteration, for m actions and n states
- Could also apply to action-value function $q_{\pi}(s, a)$ or $q_{*}(s, a)$
- Complexity $O(m^2n^2)$ per iteration

Asynchronous Dynamic Programming

- DP methods described so far used **synchronous backups**, i.e. all states are backed up in parallel, which can be unrealistic
- Asynchronous DP backs up states individually, in any order
 - For each selected state, apply the appropriate backup
 - Can significantly reduce computation
 - Guaranteed to converge if all states continue to be selected
- Three simple ideas:
 - In-place dynamic programming - only stores one copy of value function (immediately updates)
 - Prioritised sweeping – prioritizes states (e.g. with largest Bellman errors)
 - Real-time dynamic programming – updates only states relevant to the agent (e.g. the ones where rewards are being collected at a certain point)

Full-width backups vs sampling

- DP uses full-width backups
- For each backup (sync or async)
 - Every successor state and action is considered
 - Using knowledge of the MDP transitions and reward function
- DP is effective for medium-sized problems (millions of states)
- For large problems DP suffers Bellman's **curse of dimensionality**
 - Number of states grows exponentially with number of state variables
- Even one backup can be too expensive
- One alternative is to consider **sample rewards and sample transitions** – consider just a set of states/ set of possible actions; used in many real cases and algorithms

Relevant questions

- How do we know that value iteration converges to v_* ?
- Or that iterative policy evaluation converges to v_π ?
- And that policy iteration converges to v_* ?
- These questions are resolved by the ***contraction mapping theorem*** (not shown here)

Conclusions

- Classical DP algorithms are of limited utility in RL both because of their assumption of a perfect model and because of their great computational expense
- Still, they are **efficient for small scale** cases (up to millions of states) where the MDP is known or can be inferred, and they are important theoretically, as they provide an essential foundation for the understanding of the remaining methods
- Most methods can be viewed as attempts to achieve much the same effect as DP, only with **less computation** and without assuming a perfect model of the environment